Probability, Filtrations, and Adaptive Data

The Hidden Grammar of Bandit Proofs

Author: Tang

Institute: Tang's Machine Learning Blog

Date: 2026-06-18

Version: 1.0.0

Topic: Probability, Filtrations, Adaptive Data, Bandit Proofs

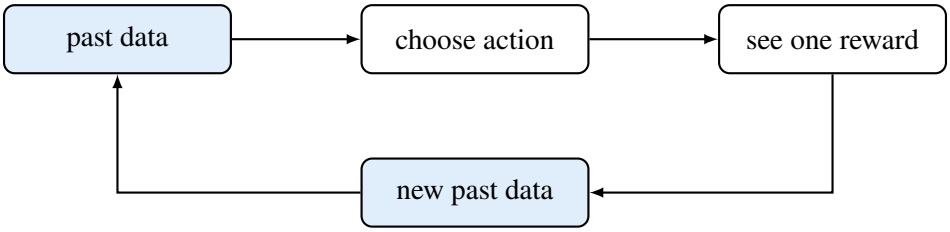
Contents

Chapter 1 The real problem: data is not waiting on a table

In supervised learning, the data set usually arrives first. We open the file, see rows of examples, and then fit a model. The model may be complicated, but the data has already been collected.

Bandit learning reverses the order. The algorithm acts first. The data appears because of the action.

That one reversal is the source of almost all probability in bandit theory.



The hard part is not that rewards are random. The hard part is that the learner chooses which randomness it will see. A bad early choice can starve an arm of data. A lucky early reward can make a bad arm look good. A confidence interval that is valid at one fixed time can become misleading if we keep peeking and stop when it looks favorable.

This note is about the grammar that prevents those mistakes.

Key idea

A bandit proof does not start with "let Ω be a probability space." It starts with a simple rule of order:

$$\text{past} \longrightarrow \text{action} \longrightarrow \text{reward}.$$

The formal probability notation is just a clean way to respect this order.

Chapter 2 Random variables: numbers before they are seen

A random variable is a number whose value has not yet been revealed.

For a Bernoulli reward, the number is either 0 or 1. If the click probability is p, then

$$X = \begin{cases} 1, & \text{with probability } p, \\ 0, & \text{with probability } 1-p. \end{cases}$$

The expectation is the long-run average value. Here there are only two possible values, so the calculation is direct:

$$\mathbb{E}[X] = 1 \cdot \mathbb{P}(X=1) + 0 \cdot \mathbb{P}(X=0) \tag{2.1}$$

$$= 1 \cdot p + 0 \cdot (1-p) \tag{2.2}$$

$$= p. \tag{2.3}$$

If we observe n independent rewards $X_1, \ldots, X_n$, the sample average is

$$\widehat{p}_n = \frac{1}{n} \sum_{s=1}^{n} X_s.$$

The sample average is random before the experiment is run. Its expectation is easy to compute:

$$\mathbb{E}[\widehat{p}_n] = \mathbb{E}\left[\frac{1}{n} \sum_{s=1}^{n} X_s\right] \tag{2.4}$$

$$= \frac{1}{n} \sum_{s=1}^{n} \mathbb{E}[X_s] \tag{2.5}$$

$$= \frac{1}{n} \sum_{s=1}^{n} p \tag{2.6}$$

$$= p. \tag{2.7}$$

This is the first comfort: the sample average points to the right target on average.

But an average can point in the right direction and still wobble. Probability inequalities measure the wobble.

Chapter 3 Error bars: turning randomness into a usable statement

For Bernoulli rewards, Hoeffding's inequality says

$$\mathbb{P}\left(|\widehat{p}_n - p| \geq \varepsilon\right) \leq 2\exp(-2n\varepsilon^2). \tag{3.1}$$

This formula is not meant to be memorized as decoration. It answers a practical question: how large should the error bar be if we want failure probability at most δ?

Start from the right-hand side and set it equal to δ:

$$2\exp(-2n\varepsilon^2) = \delta, \tag{3.2}$$

$$\exp(-2n\varepsilon^2) = \frac{\delta}{2}, \tag{3.3}$$

$$-2n\varepsilon^2 = \log\left(\frac{\delta}{2}\right), \tag{3.4}$$

$$2n\varepsilon^2 = \log\left(\frac{2}{\delta}\right), \tag{3.5}$$

$$\varepsilon^2 = \frac{\log(2/\delta)}{2n}, \tag{3.6}$$

$$\varepsilon = \sqrt{\frac{\log(2/\delta)}{2n}}. \tag{3.7}$$

So we may write the fixed-time confidence statement as

$$\mathbb{P}\left(|\widehat{p}_n - p| \leq \sqrt{\frac{\log(2/\delta)}{2n}}\right) \geq 1 - \delta. \tag{3.8}$$

Think

The phrase "fixed time" is doing real work. The statement above is about one sample size n chosen before looking at the data. It is not automatically a promise that the same error bar works after repeatedly looking at the data and stopping at a convenient time.

Chapter 4 A first warning: peeking changes the game

Suppose a coin is fair, so $p = 1/2$. If we toss it exactly 200 times and look once at the end, a standard error bar behaves roughly as expected.

But if we look after every toss and stop the moment the running average looks high, the final number is no longer an ordinary fixed-time average. We have selected a time because the noise looked favorable.

This is not a philosophical issue. It appears directly in simulation.

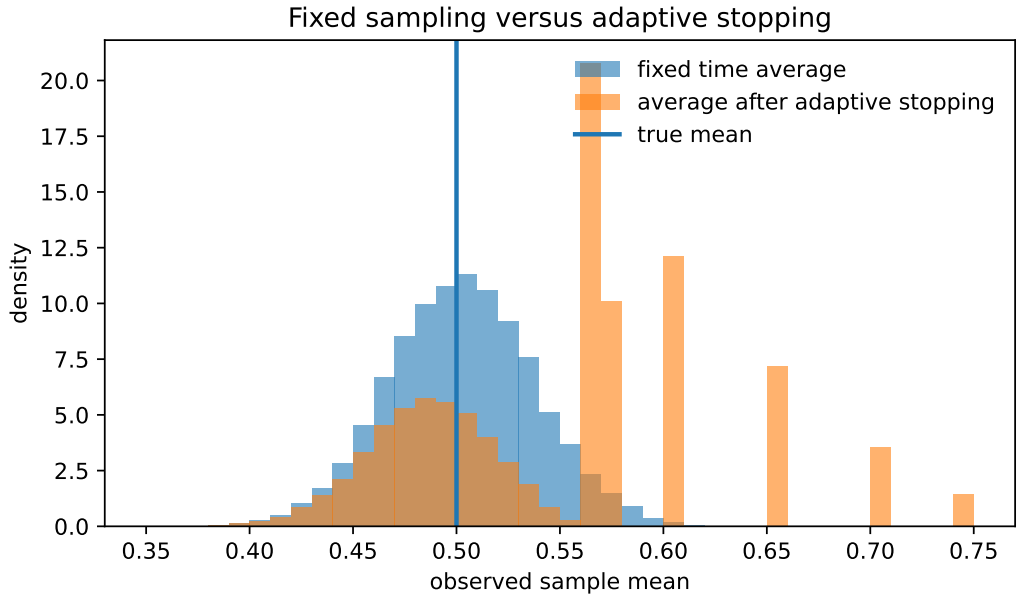


Figure 4.1: The fixed-time mean is centered near the true value 0.5. The adaptively stopped mean is shifted upward because we stop when the noise is favorable.

The same idea appears when we keep checking an error bar many times.

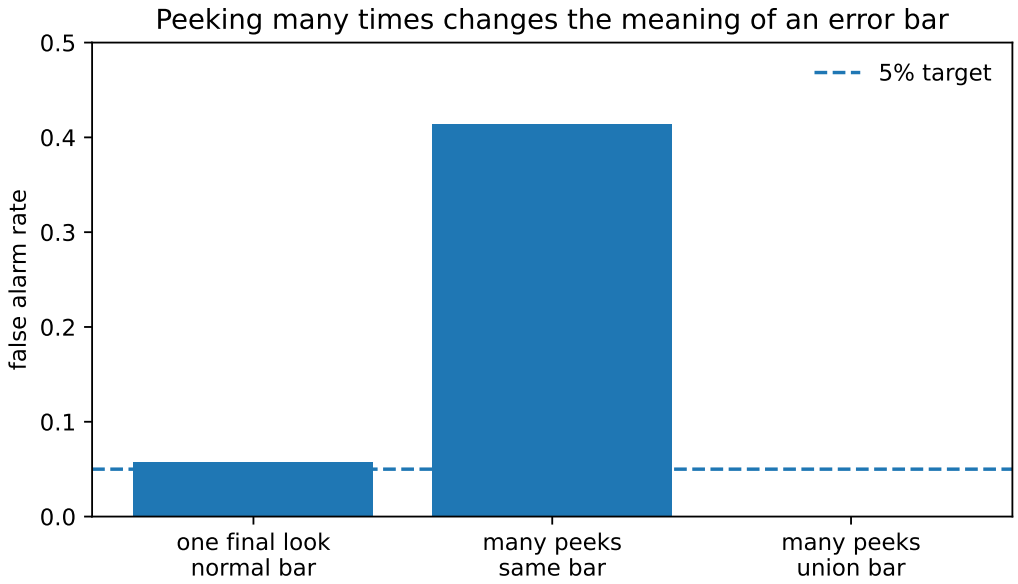


Figure 4.2: A normal-looking error bar is reasonable for one final look, but repeated peeking with the same bar creates many false alarms. A union-bound bar is crude but safe.

The experiment used $20{,}000$ runs of a fair Bernoulli process with horizon $T = 200$. The stopped rule begins checking after 20 samples and stops when the running mean first exceeds 0.56.

Quantity	Value
True Bernoulli mean	0.500
Probability of stopping early	0.555
Average fixed-time mean	0.500
Average stopped mean	0.548
One final look false alarm with normal bar	0.057
Many peeks false alarm with same normal bar	0.414
Many peeks false alarm with union bar	0.000
Post-selection average of one fixed arm	0.500
Post-selection average of empirical winner among 20 arms	0.705

Table 4.1: The important number is not the exact decimal. The important lesson is that adapting to the observed noise changes the distribution of what we report.

Chapter 5 Histories: the learner's notebook

In a bandit problem, the learner accumulates a notebook.

At the beginning of round t, the notebook contains all past actions and rewards:

$$H_{t-1} = (A_1, R_1, A_2, R_2, \ldots, A_{t-1}, R_{t-1}). \tag{5.1}$$

Then the learner chooses an action using only this notebook:

$$A_t = \pi_t(H_{t-1}). \tag{5.2}$$

Then the environment reveals one reward:

$$R_t \sim \text{reward distribution of arm } A_t. \tag{5.3}$$

Then the notebook becomes

$$H_t = (H_{t-1}, A_t, R_t). \tag{5.4}$$

A filtration is just the mathematical name for this growing notebook. We write

$$\mathcal{F}_t = \text{all information contained in } H_t. \tag{5.5}$$

So $\mathcal{F}_{t-1}$ means "everything known before choosing and observing at time t."

Key idea

A filtration is not a mysterious object. It is the learner's notebook, ordered by time. The only rule is that the algorithm may read old pages before acting, but it cannot read the next reward before choosing the next action.

Chapter 6 Conditional expectation: averaging after the past is known

The ordinary expectation $\mathbb{E}[X]$ averages before anything is known. The conditional expectation $\mathbb{E}[X \mid \mathcal{F}_{t-1}]$ averages after the past notebook has been opened.

For a bandit reward, suppose arm i has mean μ_i. If the learner chooses $A_t = i$, then

$$\mathbb{E}[R_t \mid \mathcal{F}_{t-1}, A_t = i] = \mu_i. \tag{6.1}$$

But A_t itself is chosen using $\mathcal{F}_{t-1}$, so once the past is known, A_t is also known. Hence

$$\mathbb{E}[R_t \mid \mathcal{F}_{t-1}] = \mu_{A_t}. \tag{6.2}$$

Now define the one-step noise

$$\eta_t = R_t - \mu_{A_t}. \tag{6.3}$$

Using (6.2),

$$\begin{aligned} \mathbb{E}[\eta_t \mid \mathcal{F}_{t-1}] &= \mathbb{E}[R_t - \mu_{A_t} \mid \mathcal{F}_{t-1}] && (6.4)\\ &= \mathbb{E}[R_t \mid \mathcal{F}_{t-1}] - \mu_{A_t} && (6.5)\\ &= \mu_{A_t} - \mu_{A_t} && (6.6)\\ &= 0. && (6.7) \end{aligned}$$

This tiny equation is everywhere in bandit theory. It says: after we condition on the past, the remaining reward noise is fair.

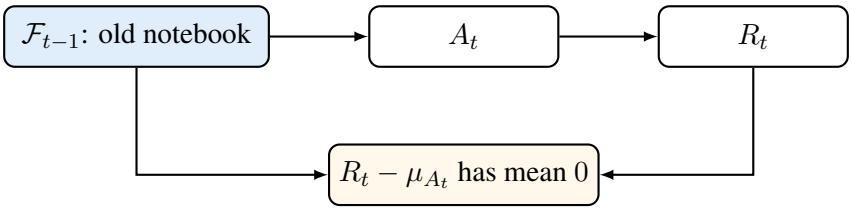
Chapter 7 Martingale differences: fair noise after conditioning

A sequence η_t satisfying

$$\mathbb{E}[\eta_t \mid \mathcal{F}_{t-1}] = 0 \tag{7.1}$$

is called a martingale difference sequence.

The name is less important than the picture. At time t, the learner may be very clever. It may choose A_t using every previous reward. But after the action is fixed, the new noise still has mean zero.



This is why adaptive data is not hopeless. The actions are adaptive, but the reward noise remains conditionally centered.

Chapter 8 Adaptive sample means

For arm i, define the number of times it has been pulled by time t:

$$N_i(t) = \sum_{s=1}^{t} \mathbf{1}\{A_s = i\}. \tag{8.1}$$

Define the empirical mean of arm i:

$$\widehat{\mu}_i(t) = \frac{\sum_{s=1}^{t} \mathbf{1}\{A_s = i\} R_s}{N_i(t)} \quad \text{when } N_i(t) > 0. \tag{8.2}$$

The numerator has two pieces:

$$\sum_{s=1}^{t} \mathbf{1}\{A_s = i\} R_s = \sum_{s=1}^{t} \mathbf{1}\{A_s = i\}(\mu_i + R_s - \mu_i) \tag{8.3}$$

$$= \mu_i \sum_{s=1}^{t} \mathbf{1}\{A_s = i\} + \sum_{s=1}^{t} \mathbf{1}\{A_s = i\}(R_s - \mu_i) \tag{8.4}$$

$$= \mu_i N_i(t) + \sum_{s=1}^{t} \mathbf{1}\{A_s = i\}(R_s - \mu_i). \tag{8.5}$$

Divide by $N_i(t)$:

$$\widehat{\mu}_i(t) - \mu_i = \frac{1}{N_i(t)} \sum_{s=1}^{t} \mathbf{1}\{A_s = i\}(R_s - \mu_i). \tag{8.6}$$

The error is still an average of centered noise terms, but the number of terms is random and chosen adaptively. That is the reason for filtrations and anytime confidence bounds.

The key centering step is

$$\mathbb{E}\left[\mathbf{1}\{A_s = i\}(R_s - \mu_i) \mid \mathcal{F}_{s-1}\right] = \mathbf{1}\{A_s = i\}\, \mathbb{E}\left[(R_s - \mu_i) \mid \mathcal{F}_{s-1}, A_s = i\right] \tag{8.7}$$

$$= \mathbf{1}\{A_s = i\} \cdot 0 \tag{8.8}$$

$$= 0. \tag{8.9}$$

The indicator $\mathbf{1}\{A_s = i\}$ is allowed because the action is chosen from the past. It is known at the moment we average over the new reward.

Think

This is the essence of adaptive data: the learner can choose which arm to sample, but it cannot choose the new random fluctuation after the arm has been chosen. That remaining fluctuation is still fair after conditioning on the past.

Chapter 9 The union bound: a simple way to be right many times

The union bound says that the probability that at least one bad event happens is at most the sum of the probabilities of the bad events.

For events $E_1, \dots, E_m$,

$$\mathbb{P}\left(\bigcup_{j=1}^{m} E_j\right) \le \sum_{j=1}^{m} \mathbb{P}(E_j). \tag{9.1}$$

A one-line proof uses indicators. For every outcome,

$$\mathbf{1}\left\{\bigcup_{j=1}^{m} E_j\right\} \le \sum_{j=1}^{m} \mathbf{1}\{E_j\}. \tag{9.2}$$

Take expectations:

$$\mathbb{P}\left(\bigcup_{j=1}^{m} E_j\right) = \mathbb{E}\left[\mathbf{1}\left\{\bigcup_{j=1}^{m} E_j\right\}\right] \tag{9.3}$$

$$\le \mathbb{E}\left[\sum_{j=1}^{m} \mathbf{1}\{E_j\}\right] \tag{9.4}$$

$$= \sum_{j=1}^{m} \mathbb{E}[\mathbf{1}\{E_j\}] \tag{9.5}$$

$$= \sum_{j=1}^{m} \mathbb{P}(E_j). \tag{9.6}$$

Now suppose we want confidence intervals for K arms and T possible times. There are at most KT things we want to be simultaneously correct about.

Give each one failure probability

$$\delta' = \frac{\delta}{KT}. \tag{9.7}$$

For each arm-time pair, Hoeffding gives

$$\mathbb{P}\left(|\widehat{\mu}_i(t) - \mu_i| > \sqrt{\frac{\log(2/\delta')}{2N_i(t)}}\right) \le \delta'. \tag{9.8}$$

Substitute $\delta' = \delta/(KT)$:

$$\sqrt{\frac{\log(2/\delta')}{2N_i(t)}} = \sqrt{\frac{\log(2KT/\delta)}{2N_i(t)}}. \tag{9.9}$$

Then the union bound says that all these intervals are correct together with probability at least $1 - \delta$.

This is crude. Modern analyses often use sharper time-uniform tools. But the union-bound version is the first clean proof pattern, and it already explains why logarithms appear in bandit regret bounds.

Chapter 10 From probability to UCB

UCB is built from one sentence:

Choose the arm whose plausible optimistic value is largest.

At time t, define the radius

$$\mathrm{rad}_i(t) = \sqrt{\frac{\log(2KT/\delta)}{2N_i(t)}}. \tag{10.1}$$

The optimistic index is

$$\mathrm{UCB}_i(t) = \widehat{\mu}_i(t) + \mathrm{rad}_i(t). \tag{10.2}$$

The algorithm chooses

$$A_{t+1} \in \arg\max_i \mathrm{UCB}_i(t). \tag{10.3}$$

The confidence event is

$$\mathcal{G} = \{\forall i, \forall t : \ |\widehat{\mu}_i(t) - \mu_i| \le \mathrm{rad}_i(t)\}\,. \tag{10.4}$$

On $\mathcal{G}$, every empirical mean is close to its truth at every relevant time. That single event drives the UCB proof.

10.1 The pull-count argument, line by line

Let i^* be an optimal arm and let i be a suboptimal arm. Define the gap

$$\Delta_i = \mu_{i^*} - \mu_i > 0. \tag{10.5}$$

Suppose UCB pulls arm i at some time. Since UCB chose i instead of i^*,

$$\widehat{\mu}_i + \mathrm{rad}_i \ge \widehat{\mu}_{i^*} + \mathrm{rad}_{i^*}\,. \tag{10.6}$$

On the good event $\mathcal{G}$,

$$\widehat{\mu}_{i^*} + \mathrm{rad}_{i^*} \ge \mu_{i^*}. \tag{10.7}$$

Combining (10.6) and (10.7),

$$\widehat{\mu}_i + \mathrm{rad}_i \ge \mu_{i^*}. \tag{10.8}$$

Again on $\mathcal{G}$,

$$\widehat{\mu}_i \le \mu_i + \mathrm{rad}_i\,. \tag{10.9}$$

Substitute (10.9) into (10.8):

$$\mu_i + \mathrm{rad}_i + \mathrm{rad}_i \ge \mu_{i^*}, \tag{10.10}$$

$$2\,\mathrm{rad}_i \ge \mu_{i^*} - \mu_i, \tag{10.11}$$

$$2\,\mathrm{rad}_i \ge \Delta_i, \tag{10.12}$$

$$\mathrm{rad}_i \ge \frac{\Delta_i}{2}. \tag{10.13}$$

Now plug in the radius:

$$\sqrt{\frac{\log(2KT/\delta)}{2N_i(t)}} \geq \frac{\Delta_i}{2}, \tag{10.14}$$

$$\frac{\log(2KT/\delta)}{2N_i(t)} \geq \frac{\Delta_i^2}{4}, \tag{10.15}$$

$$4\log(2KT/\delta) \geq 2N_i(t)\Delta_i^2, \tag{10.16}$$

$$N_i(t) \leq \frac{2\log(2KT/\delta)}{\Delta_i^2}. \tag{10.17}$$

Depending on the exact convention for the radius, constants change. The message does not change: a bad arm can only be pulled many times if its uncertainty radius is still large. Once it has been sampled enough, its upper confidence bound drops below the optimal arm's plausible value.

Key idea

The UCB proof is not magic. It is a bookkeeping argument:

bad arm chosen $\Rightarrow$ its uncertainty must still be large $\Rightarrow$ it has not been sampled too often.

Chapter 11 Post-selection bias: the quiet enemy

Another common mistake is to look at many random estimates and then trust the largest one as if it had been chosen in advance.

Imagine 20 arms. In this experiment they are all identical: every arm has true mean 0.5. We sample each arm 20 times and select the one with the largest empirical mean.

The selected arm looks much better than 0.5, not because it is truly better, but because it won a noise contest.

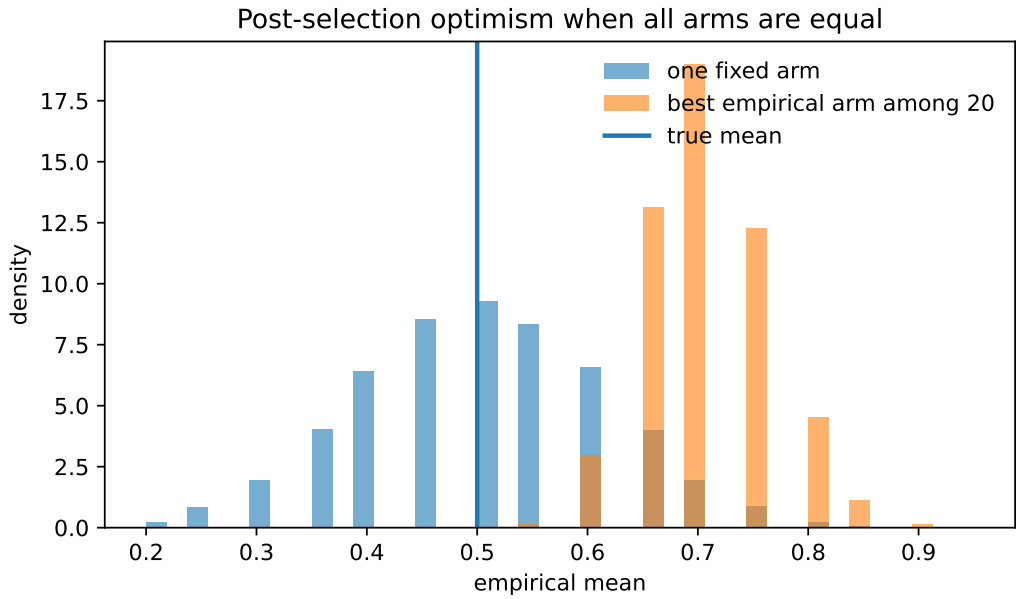


Figure 11.1: When all arms have the same true mean, the best empirical arm is still biased upward. Selection turns noise into apparent signal.

This is why bandit algorithms need explicit exploration rules. Greedy selection tends to chase the current winner. Confidence-based and posterior-based algorithms try to remember that the current winner may be winning only because the data is thin.

Chapter 12 Code implementation

The simulation is deliberately small. It is not a benchmark. It is a microscope.

The first part compares one fixed-time average with an adaptively stopped average.

Adaptive stopping experiment.

```
for r in range(cfg.runs):
    x = rng.binomial(1, cfg.p, size=cfg.fixed_n)
    s = np.cumsum(x)
    t = np.arange(1, cfg.fixed_n + 1)
    m = s / t

    fixed_means[r] = m[-1]

    normal_radius = 1.96 * np.sqrt(0.25 / t)
    union_r = union_radius(t, cfg.alpha, cfg.fixed_n)
    final_normal_false_alarm[r] = abs(m[-1] - cfg.p) > normal_radius[-1]
    anytime_normal_false_alarm[r] = np.any(np.abs(m - cfg.p) > normal_radius)
    anytime_union_false_alarm[r] = np.any(np.abs(m - cfg.p) > union_r)

    eligible = np.where((t >= cfg.stop_start) & (m >= cfg.threshold))[0]
    if eligible.size > 0:
        j = int(eligible[0])
        stopped_times[r] = j + 1
        stopped_means[r] = m[j]
        stopped_by_rule[r] = True
    else:
        stopped_times[r] = cfg.fixed_n
        stopped_means[r] = m[-1]
        stopped_by_rule[r] = False
```

The second part shows post-selection bias.

Post-selection experiment.

```
# All arms have the same true mean.  Any winner is a winner only because of noise.
samples = rng.binomial(1, cfg.p, size=(cfg.runs, cfg.k_arms, cfg.n_per_arm))
means = samples.mean(axis=2)
selected = means.max(axis=1)
ordinary = means[:, 0]
```

The full script is included in the appendix and in the accompanying simulation file.

Chapter 13 What this chapter prepares

The next bandit algorithms will use the same grammar repeatedly.

For UCB, the key object is a confidence event that holds across arms and times.

For Thompson sampling, the key object is a posterior distribution updated by the same history $\mathcal{F}_t$.

For best-arm identification, the key object is a stopping time: a random time at which the algorithm decides it has enough evidence.

For batched bandits, the key object is restricted adaptivity: the notebook is updated only at batch boundaries.

So probability is not a decorative layer. It is the language that keeps time, information, and evidence in the correct order.

Key idea

The deepest lesson is simple: in bandits, data has a timestamp. A proof is correct only if it respects what was known before the action and what was revealed after the action.

Appendix A. Symbol Table

Symbol	Meaning
A_t	action chosen at round t
R_t	reward observed after choosing A_t
H_t	history or notebook after round t
$\mathcal{F}_t$	filtration: all information contained in H_t
μ_i	true mean reward of arm i
$N_i(t)$	number of pulls of arm i by time t
$\widehat{\mu}_i(t)$	empirical mean of arm i by time t
Δ_i	suboptimality gap $\mu_{i^*} - \mu_i$
η_t	centered reward noise $R_t - \mu_{A_t}$
$\mathrm{rad}_i(t)$	confidence radius for arm i at time t
$\mathcal{G}$	good event on which all confidence intervals are simultaneously correct

Appendix B. Core Formulas

$$\mathbb{E}[X] = \sum_{x} x\,\mathbb{P}(X = x), \tag{13.1}$$

$$\widehat{p}_n = \frac{1}{n}\sum_{s=1}^{n} X_s, \tag{13.2}$$

$$\mathbb{P}\left(|\widehat{p}_n - p| \geq \varepsilon\right) \leq 2\exp(-2n\varepsilon^2), \tag{13.3}$$

$$\varepsilon_n(\delta) = \sqrt{\frac{\log(2/\delta)}{2n}}, \tag{13.4}$$

$$H_t = (A_1, R_1, \ldots, A_t, R_t), \tag{13.5}$$

$$\mathbb{E}[R_t \mid \mathcal{F}_{t-1}] = \mu_{A_t}, \tag{13.6}$$

$$\mathbb{E}[R_t - \mu_{A_t} \mid \mathcal{F}_{t-1}] = 0, \tag{13.7}$$

$$N_i(t) = \sum_{s=1}^{t} \mathbf{1}\{A_s = i\}, \tag{13.8}$$

$$\widehat{\mu}_i(t) - \mu_i = \frac{1}{N_i(t)}\sum_{s=1}^{t} \mathbf{1}\{A_s = i\}(R_s - \mu_i), \tag{13.9}$$

$$\mathbb{P}\left(\bigcup_{j} E_j\right) \leq \sum_{j} \mathbb{P}(E_j), \tag{13.10}$$

$$\mathrm{UCB}_i(t) = \widehat{\mu}_i(t) + \sqrt{\frac{\log(2KT/\delta)}{2N_i(t)}}. \tag{13.11}$$

Appendix C. Full Simulation Code

Listing 13.1: Full simulation code.

```
"""
Simulation for the lecture note:
Probability, Filtrations, and Adaptive Data.

The goal is not to build a large benchmark.  The goal is to make three
probability ideas visible:

1. A sample average is reliable at one fixed time.
2. Looking many times with the same fixed-time error bar creates false discoveries.
3. Selecting the largest empirical mean among many arms creates upward bias.

Run:
    python probability-filtrations-simulation.py
"""
from __future__ import annotations

import os
from dataclasses import dataclass
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

OUT = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(OUT, "figures")
os.makedirs(FIG, exist_ok=True)

rng = np.random.default_rng(7)


@dataclass
class Config:
    p: float = 0.50
    runs: int = 20000
    fixed_n: int = 200
    stop_start: int = 20
    threshold: float = 0.56
    alpha: float = 0.05
    k_arms: int = 20
    n_per_arm: int = 20


def hoeffding_radius(n: np.ndarray | float, alpha: float) -> np.ndarray | float:
    """Fixed-time two-sided Hoeffding radius for Bernoulli rewards in [0,1]."""
```

```python
    return np.sqrt(np.log(2.0 / alpha) / (2.0 * n))


def union_radius(n: np.ndarray | float, alpha: float, T: int) -> np.ndarray | float:
    """A simple anytime radius obtained by a union bound over t=1,...,T."""
    return np.sqrt(np.log(2.0 * T / alpha) / (2.0 * n))


def simulate_fixed_and_stopped(cfg: Config):
    fixed_means = np.empty(cfg.runs)
    stopped_means = np.empty(cfg.runs)
    stopped_times = np.empty(cfg.runs, dtype=int)
    stopped_by_rule = np.empty(cfg.runs, dtype=bool)
    final_normal_false_alarm = np.empty(cfg.runs, dtype=bool)
    anytime_normal_false_alarm = np.empty(cfg.runs, dtype=bool)
    anytime_union_false_alarm = np.empty(cfg.runs, dtype=bool)

    for r in range(cfg.runs):
        x = rng.binomial(1, cfg.p, size=cfg.fixed_n)
        s = np.cumsum(x)
        t = np.arange(1, cfg.fixed_n + 1)
        m = s / t

        fixed_means[r] = m[-1]

        normal_radius = 1.96 * np.sqrt(0.25 / t)
        union_r = union_radius(t, cfg.alpha, cfg.fixed_n)
        final_normal_false_alarm[r] = abs(m[-1] - cfg.p) > normal_radius[-1]
        anytime_normal_false_alarm[r] = np.any(np.abs(m - cfg.p) > normal_radius)
        anytime_union_false_alarm[r] = np.any(np.abs(m - cfg.p) > union_r)

        eligible = np.where((t >= cfg.stop_start) & (m >= cfg.threshold))[0]
        if eligible.size > 0:
            j = int(eligible[0])
            stopped_times[r] = j + 1
            stopped_means[r] = m[j]
            stopped_by_rule[r] = True
        else:
            stopped_times[r] = cfg.fixed_n
            stopped_means[r] = m[-1]
            stopped_by_rule[r] = False

    rad_fixed = hoeffding_radius(cfg.fixed_n, cfg.alpha)
    fixed_cover = np.mean(np.abs(fixed_means - cfg.p) <= rad_fixed)

    rad_stopped_naive = hoeffding_radius(stopped_times, cfg.alpha)
    stopped_cover_naive = np.mean(np.abs(stopped_means - cfg.p) <= rad_stopped_naive)

```

```
    rad_stopped_union = union_radius(stopped_times, cfg.alpha, cfg.fixed_n)
    stopped_cover_union = np.mean(np.abs(stopped_means - cfg.p) <= rad_stopped_union)

    return {
        "fixed_means": fixed_means,
        "stopped_means": stopped_means,
        "stopped_times": stopped_times,
        "stopped_by_rule": stopped_by_rule,
        "fixed_cover": fixed_cover,
        "stopped_cover_naive": stopped_cover_naive,
        "stopped_cover_union": stopped_cover_union,
        "final_normal_false_alarm": np.mean(final_normal_false_alarm),
        "anytime_normal_false_alarm": np.mean(anytime_normal_false_alarm),
        "anytime_union_false_alarm": np.mean(anytime_union_false_alarm),
        "stop_rate": np.mean(stopped_by_rule),
        "avg_stop_time": np.mean(stopped_times),
        "fixed_mean_avg": np.mean(fixed_means),
        "stopped_mean_avg": np.mean(stopped_means),
    }


def simulate_post_selection(cfg: Config):
    # All arms have the same true mean.  Any winner is a winner only because of noise.
    samples = rng.binomial(1, cfg.p, size=(cfg.runs, cfg.k_arms, cfg.n_per_arm))
    means = samples.mean(axis=2)
    selected = means.max(axis=1)
    ordinary = means[:, 0]
    return {
        "selected_means": selected,
        "ordinary_means": ordinary,
        "selected_mean_avg": float(np.mean(selected)),
        "ordinary_mean_avg": float(np.mean(ordinary)),
        "selection_bias": float(np.mean(selected) - cfg.p),
    }


def make_plots(cfg: Config, fs, ps):
    plt.figure(figsize=(6.6, 4.0))
    bins = np.linspace(0.35, 0.75, 41)
    plt.hist(fs["fixed_means"], bins=bins, alpha=0.6, density=True, label="fixed time
    average")
    plt.hist(fs["stopped_means"], bins=bins, alpha=0.6, density=True, label="average after
    adaptive stopping")
    plt.axvline(cfg.p, linewidth=2, label="true mean")
    plt.xlabel("observed sample mean")
    plt.ylabel("density")
    plt.title("Fixed sampling versus adaptive stopping")
    plt.legend(frameon=False)
```

```
    plt.tight_layout()
    plt.savefig(os.path.join(FIG, "fixed_vs_stopped_means.pdf"))
    plt.savefig(os.path.join(FIG, "fixed_vs_stopped_means.png"), dpi=200)
    plt.close()

    labels = ["one final look\nnormal bar", "many peeks\nsame bar", "many peeks\nunion bar"]
    vals = [fs["final_normal_false_alarm"], fs["anytime_normal_false_alarm"], fs["
    anytime_union_false_alarm"]]
    plt.figure(figsize=(6.4, 3.8))
    plt.bar(labels, vals)
    plt.axhline(0.05, linestyle="--", linewidth=1.5, label="5% target")
    plt.ylim(0.0, 0.50)
    plt.ylabel("false alarm rate")
    plt.title("Peeking many times changes the meaning of an error bar")
    plt.legend(frameon=False)
    plt.tight_layout()
    plt.savefig(os.path.join(FIG, "false_alarm_comparison.pdf"))
    plt.savefig(os.path.join(FIG, "false_alarm_comparison.png"), dpi=200)
    plt.close()

    plt.figure(figsize=(6.6, 4.0))
    bins = np.linspace(0.2, 0.95, 41)
    plt.hist(ps["ordinary_means"], bins=bins, alpha=0.6, density=True, label="one fixed arm"
    )
    plt.hist(ps["selected_means"], bins=bins, alpha=0.6, density=True, label="best empirical
     arm among 20")
    plt.axvline(cfg.p, linewidth=2, label="true mean")
    plt.xlabel("empirical mean")
    plt.ylabel("density")
    plt.title("Post-selection optimism when all arms are equal")
    plt.legend(frameon=False)
    plt.tight_layout()
    plt.savefig(os.path.join(FIG, "post_selection_bias.pdf"))
    plt.savefig(os.path.join(FIG, "post_selection_bias.png"), dpi=200)
    plt.close()


def main():
    cfg = Config()
    fs = simulate_fixed_and_stopped(cfg)
    ps = simulate_post_selection(cfg)
    make_plots(cfg, fs, ps)

    rows = [
        {"quantity": "true Bernoulli mean", "value": cfg.p},
        {"quantity": "runs", "value": cfg.runs},
        {"quantity": "fixed sample size", "value": cfg.fixed_n},
        {"quantity": "adaptive stop threshold", "value": cfg.threshold},
```

```
        {"quantity": "probability of stopping early", "value": fs["stop_rate"]},
        {"quantity": "average stopping time", "value": fs["avg_stop_time"]},
        {"quantity": "average fixed-time mean", "value": fs["fixed_mean_avg"]},
        {"quantity": "average stopped mean", "value": fs["stopped_mean_avg"]},
        {"quantity": "fixed-time Hoeffding coverage", "value": fs["fixed_cover"]},
        {"quantity": "stopped-time naive Hoeffding coverage", "value": fs["
    stopped_cover_naive"]},
        {"quantity": "stopped-time union-bound Hoeffding coverage", "value": fs["
    stopped_cover_union"]},
        {"quantity": "one final look false alarm with normal bar", "value": fs["
    final_normal_false_alarm"]},
        {"quantity": "many peeks false alarm with same normal bar", "value": fs["
    anytime_normal_false_alarm"]},
        {"quantity": "many peeks false alarm with union bar", "value": fs["
    anytime_union_false_alarm"]},
        {"quantity": "post-selection average of one fixed arm", "value": ps["
    ordinary_mean_avg"]},
        {"quantity": "post-selection average of empirical winner", "value": ps["
    selected_mean_avg"]},
        {"quantity": "post-selection bias", "value": ps["selection_bias"]},
    ]
    df = pd.DataFrame(rows)
    df.to_csv(os.path.join(OUT, "results.csv"), index=False)
    print(df.to_string(index=False))


if __name__ == "__main__":
    main()
```

References

[1] P. Auer, N. Cesa-Bianchi, and P. Fischer. “Finite-time analysis of the multiarmed bandit problem”. In: *Machine Learning* 47 (2002), pp. 235–256.

[2] S. Bubeck and N. Cesa-Bianchi. “Regret analysis of stochastic and nonstochastic multi-armed bandit problems”. In: *Foundations and Trends in Machine Learning* 5.1 (2012), pp. 1–122.

[3] Steven R. Howard et al. “Time-uniform Chernoff bounds via nonnegative supermartingales”. In: *Probability Surveys* 17 (2020), pp. 257–317.

[4] T. Lattimore and C. Szepesvari. *Bandit Algorithms*. Cambridge University Press, 2020.